

# Computer Organization and Assembly language

## Final Exam

Date: May 20<sup>th</sup> 2025

Course Instructor(s)

Mr. Aashir Mahboob and Dr. Muhammad Adeel

Total Time (Hrs): 3

Total Marks: 100

Total Questions: 8

Roll No

Section

Student Signature

Do not write below this line

## Solution

**CLO # 2: Identify use of processor register and addressing modes in various instructions for solving arithmetic data transfer and conditional processing**

**Q1: Briefly answer each of the following:**

**[10 x 2 = 20 Marks]**

- I. What will be the contents of AX and DX after the following operation?

```
MOV DX,0  
MOV AX,222h  
MOV CX,100h  
MUL CX
```

**AX = 2200 h , DX= 0002 h**

- II. What will be the contents of BX after the following instructions execute?

```
MOV BX,50H  
CLC  
MOV AX,6  
SBB BX,AX
```

**BX= 4A h**

- III. What is meant by Divide overflow? Give example instruction

**Divide overflow happens when the quotient is too large to fit into the destination. (or DIV by Zero).**

```
mov ax, 1000h  
mov bl, 10h  
div bl
```

- IV. Why Sign-extension is necessary for signed division? Explain with example instruction.

**Because, if sign is not extended, the quotient and remainder will give wrong values.**

# National University of Computer and Emerging Sciences

## Karachi Campus

```
.code
mov     dx,0
mov     ax,wordVal          ; DX:AX = 0000FF9Bh
mov     bx,2                 ; BX is the divisor
idiv    bx                   ; divide DX:AX by BX (signed
                             operation)
```

**Here, both answers will be positive, despite using Signed Division.**

- V. Using SUB and Shift instructions, multiply a number stored in EAX register by 27d. **(Write code)**  
**MOV EBX,EAX**  
**MOV ECX, EAX**  
**SHL EAX, 5 ; 32**  
**SHL EBX, 2 ; 4**  
**SUB EAX,EBX ; 32-4= 28**  
**SUB EAX, ECX ; 28-1 =27**
- VI. Differentiate between TEST and AND Instruction.  
**AND performs logical AND operation between 2 operands, updates destination operand and updates Status Flags, While TEST is non-destructive AND, It only updates Flags.**
- VII. Elaborate through an example instruction, how does ADDR differ from OFFSET directive?  
**Both ADDR and OFFSET directive gives address of a memory location, the only difference is that, we can only use ADDR with INVOKE only. Other than it, it generates an error.**  
**Invoke mysub , ADDR myWord ; procedure called with address as a parameter**  
**MOV esi , ADDR myArray ; Error**
- VIII. Explain the difference between CMP and SUB instruction.  
**SUB performs Arithmetic Subtraction operation between 2 operands, updates destination operand and updates Status Flags, While CMP is non-destructive SUB, It only updates Flags.**
- IX. Elaborate through an example, the purpose of Direction Flag?  
**Direction Flag is use to set direction of String Read during String primitive instructions.**  
**CLD means DF=0, string will be read left to right with increment in ESI and EDI.**  
**STD means DF=1, String will be read Right to Left with decrement in ESI and EDI.**
- X. Explain the difference between RET and RET 8 instruction.  
**RET only removes return address from Stack means ESP is only incremented by 4 Bytes, With RET 8 removes Return Address along with two DWORD parameters from STACK resulting in ESP incremented by 12 Bytes.**

### ***CLO # 3: Apply translation of machine instructions into binary code and vice versa***

**Q2:** Convert the following independent Assembly Language instructions to Machine Language code – give your answers in hexadecimal **(binary answers will not be graded):** [5 x 2 =10 Marks]

- |      |                              |                      |
|------|------------------------------|----------------------|
| I.   | MOVZX CX, VAR1 ; Var1= 19h → | <b>0F B7 0E H</b>    |
| II.  | POP 0ABCD [BX] →             | <b>8F 87 CD AB H</b> |
| III. | MOV BX, DX →                 | <b>89 D3 H</b>       |
| IV.  | INC [BP][DI]+ 3F →           | <b>FF 43 3F H</b>    |
| V.   | RCL [BX] , 2 →               | <b>C1 17 3 H</b>     |

### ***CLO # 3: Apply translation of machine instructions into binary code and vice versa***

# National University of Computer and Emerging Sciences

## Karachi Campus

**Q3:** Convert the following hexadecimal machine codes to assembly language Instructions. State what each of the byte fields mean: [5 x 2 =10 Marks]

- I. 03 84 1B 2A → **ADD AX, [SI +2A1B]**
- II. 2B 1D → **SUB BX,[DI]**
- III. F6 D3 → **NOT BL**
- IV. 56 → **PUSH SI**
- V. 0B 0A → **OR CX,[BP][SI]**

**CLO # 2: Create basic assembly code using different type of addressing modes in x86 & RISC ISAs to solve simple-moderate problems**

**Q4:** The following data have been received from a real-time measuring device and is stored in an array Array1. You are required to write an assembly language program with the corresponding data definition directives that would extract the data items and store them in WORD type variables named Day, Month, Year, Wind Speed, Pressure and Humidity. *You can only use instructions SHR and SHL to isolate data. Suppose data is 76548945h.* [10 Marks]

| 5 bits | 4 bits | 7 bits | 6 bits     | 5 bits   | 5 bits   |
|--------|--------|--------|------------|----------|----------|
| Day    | Month  | Year   | Wind Speed | Pressure | Humidity |
|        |        |        |            |          |          |

**.data**

**Day WORD ?**

**Month WORD ?**

**Year WORD ?**

**WindSpeed WORD ?**

**Pressure WORD ?**

**Humidity WORD ?**

**Array1 DWORD 76548945h**

**.code**

**Main Proc**

**MOV EAX,Array1**

**MOV EBX, EAX**

**SHR EAX, 27 ; Days are isolated**

**MOV DAY, AX**

**MOV EAX, EBX**

**SHL EAX, 5**

**SHR EAX, 28 ; Month is isolated**

**MOV Month , AX**

**MOV EAX, EBX**

**SHL EAX, 9**

**SHR EAX, 25 ; Year is isolated**

**MOV Year , AX**

**MOV EAX, EBX**

**SHL EAX, 16**

**SHR EAX, 26 ; WindSpeed is isolated**

**MOV WindSpeed , AX**

# National University of Computer and Emerging Sciences

## Karachi Campus

```
MOV EAX,EBX
SHL EAX, 22
SHR EAX, 27 ; Pressure is isolated
MOV Pressure, AX
MOV EAX,EBX
SHL EAX,27
SHR EAX, 27 ; Humidity is isolated
MOV Humidity , AX
Main ENDP
END main.
```

**CLO # 2: Create basic assembly code using different type of addressing modes in x86 & RISC ISAs to solve simple-moderate problems**

**Q5:** Write an assembly language procedure that adds the given two 128-bit numbers. Assume a 32-bit architecture. Numbers should be accessed by their addresses. Result should be stored in a memory location. Procedure should be called from MAIN PROC. **[10 Marks]**

```
Val1    QWORD    2 DUP(1234567812345678h)
Val2    QWORD    2 DUP(8765432187654321h)
Result  QWORD    3 DUP (0)
```

**.code**

**main PROC**

**mov esi, OFFSET val1**

**mov edi,OFFSET val2**

**mov ebx, OFFSET Result**

**mov ecx,4 ; number of doublewords**

**call Extended\_ADD**

**mov esi, offset Result**

**mov ebx,4**

**mov ecx, LENGTHOF Result**

**call DumpMem**

**exit**

**main ENDP**

**Extended\_Add Proc**

**Pushad**

**mov eax,0**

**clc**

**L1:**

**mov eax,[esi] ; get the first integer**

**ADC eax,[edi] ; Adding the second integer**

**pushfd ; save the Carry flag**

**mov [ebx],eax ; store partial result**

**add esi,4 ; advance all 3 pointers**

**add edi,4**

**add ebx,4**

**popfd ; restore the Carry flag**

**loop L1 ; repeat the loop**

**; repeat the loop**

**ADC word ptr [ebx],0 ; left over carry**

**popad**

# National University of Computer and Emerging Sciences

## Karachi Campus

ret  
Extended\_Add ENDP  
End main

**CLO # 2: Create basic assembly code using different type of addressing modes in x86 & RISC ISAs to solve simple-moderate problems**

**Q6:** Using String Primitive instructions, write an assembly language program that reads an integer from user, multiplies it with each element stored in given array and updates the given array. **[10 Marks]**

**Example:**

```
.data
Array1 BYTE 1,2,3,4,5,6,7,8,9,10
.code
Enter a Number: 7 ; Assuming user enters 7.
Updated Array is: 7,14,21,28,35,42,49,56,63,70
```

**INCLUDE Irvine32.inc**

```
.data
Array1 BYTE 1,2,3,4,5,6,7,8,9,10
Msg1 BYTE " Enter a Number " ,0
Msg2 BYTE " Updated Array is : " , 0
```

```
.code
Main PROC
Cld
MOV EDX, OFFSET Msg1
Call Writestring
Call Readint
MOV BL, AL
MOV ESI, OFFSET Array1
MOV EDI,ESI
MOV ECX, LENGTHOF Array1
L1: LODSB
MUL BL
STOSB
LOOP L1
MOV EDX, OFFSET Msg2
Call WriteString
MOV ESI, OFFEST Array1
MOV EBX, 1
MOV ECX, LENGHTOF Array1
Call DumpMem
```

```
Main ENDP
END Main
```

# National University of Computer and Emerging Sciences

Karachi Campus

**CLO # 4: Illustrate use of stack during a parametrized function/procedure call that uses local variables.**

**Q7:** Write an assembly language program using advance procedures, that finds the maximum out of three parameters passed by values on STACK. Parameters should be accessed by names and procedure, should be INVOKED from Main. Maximum value should be displayed on console after returning from procedure and STACK should be cleaned. Draw Runtime STACK also. **[10 Marks]**

**Maximum PROC, val1:DWORD,val2:DWORD,val3:DWORD**

**.data**

**Num1 DWORD 5**

**Num2 DWORD 6**

**Num3 DWORD 7**

**.code**

**Main PROC**

**INVOKE Maximum , Num1,Num2,Num3**

**Call Writeint**

**Main ENDP**

**Maximum PROC,**

**Val1:DWORD, Val2:DWORD,Val3:DWORD**

**MOV EAX, Val1**

**CMP EAX, Val2**

**JGE L1**

**MOV EAX, Val2**

**L1: CMP EAX, Val3**

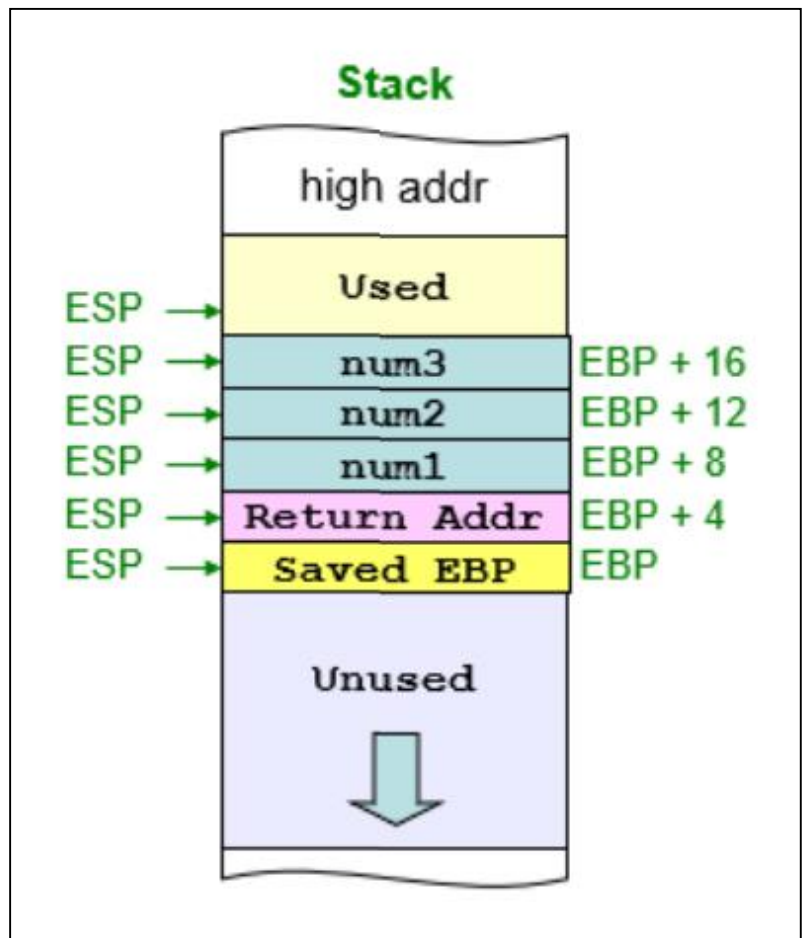
**JGE L2**

**MOV EAX, Val3**

**L2: RET**

**Maximum ENDP**

**END Main**



# National University of Computer and Emerging Sciences

Karachi Campus

**CLO # 2: Create basic assembly code using different type of addressing modes in x86 & RISC ISAs to solve simple-moderate problems**

**Q8(a)** : Suppose you are given an integer that requires 32 bits to store. You are asked to find whether its binary representation has an odd or even number of 1's. Write a program to read a positive integer from the user and outputs whether it contains an odd or even number 1's. **[ 10 + 10 = 20 Marks]**

**For example:**

Enter a Number: 6

*; 6 is entered by user.*

The Entered number has even number of 1's *; 6=0110b in binary , so even number of 1's*

**Include Irvine32.inc**

**.data**

**buffer dword ?**

**msg1 byte "Integer contains odd no. of 1s",0**

**msg2 byte "Integer contains even number of 1s",0**

**msg3 byte "Enter 32 bit number",0**

**.code**

**main PROC**

**mov edx, offset msg3**

**call writestring**

**call Readint**

**mov buffer, eax**

**call crlf**

**mov ecx, 32**

**mov ebx,0**

**mov esi, offset buffer**

**L1:**

**shl eax,1**

**jnc L2**

**add ebx, 1**

**L2:**

**inc esi**

**loop L1**

**mov edx,0**

**mov eax, ebx**

**mov ebx, 2**

**div ebx**

**cmp edx, 0**

**jnz L3**

**mov edx, offset msg2**

**jmp L4**

**L3:**

**mov edx, offset msg1**

**L4:**

**call writestring**

**exit**

**main ENDP**

# National University of Computer and Emerging Sciences

## Karachi Campus

### End main

#### Q8(b) :

In the following code sequence, show the value of destination register (in hexadecimal format only) after each shift or rotate instruction has executed:

```
MOV CL,2
MOV AL, 7Ch
MOV BL , C7h
SHL AL, CL      ; a= F0 h
SHR BL,CL       ; b= 31 h
INC CL
SAR BL,CL       ; c= 06 h
ROL AL,CL       ; d= C3 h
CLC
DEC CL
RCL AL,CL       ; e= 0D h
STC
RCR BL,CL       ; f= 41 h
SHRD AL,BL,2    ; g= 43 h
SHLD BL,AL,2    ; h= 05 h
```

===== Good Luck =====

| Instructions                                 | Opcode                             | Instructions               | Opcode                    |
|----------------------------------------------|------------------------------------|----------------------------|---------------------------|
| MOV reg, reg<br>MOV reg, mem<br>MOV mem, reg | 100010dw oorrmmm<br>disp           | RCL reg, 1<br>RCL mem,1    | 1101000w oo010mmm<br>disp |
| MOV reg, imm                                 | 1011wrrr data                      | RCR reg, 1<br>RCR mem,1    | 1101000w oo011mmm<br>disp |
| MOV mem, imm                                 | 1100011w oo000mmm<br>disp data     | ROL reg, 1<br>ROL mem,1    | 1101000w oo000mmm<br>disp |
| MOVSX reg, reg<br>MOVSX reg, mem             | 00001111 1011111w<br>oorrrmmm disp | ROR reg, 1<br>ROR mem,1    | 1101000w oo001mmm<br>disp |
| MOVZX reg, reg<br>MOVZX reg, mem             | 00001111 1011011w<br>oorrrmmm disp | RCL reg, CL<br>RCL mem, CL | 1101001w oo010mmm<br>disp |
| MUL reg<br>MUL mem                           | 1111011w oo100mmm<br>disp          | RCR reg, CL<br>RCR mem, CL | 1101001w oo011mmm<br>disp |
| NEG reg<br>NEG mem                           | 1111011w oo011mmm<br>disp          | ROL reg, CL<br>ROL mem, CL | 1101001w oo000mmm<br>disp |

# National University of Computer and Emerging Sciences

## Karachi Campus

|                                           |                           |                                              |                                |
|-------------------------------------------|---------------------------|----------------------------------------------|--------------------------------|
| NOT reg<br>NOT mem                        | 1111011w oo010mmm<br>disp | ROR reg, CL<br>ROR mem, CL                   | 1101001w oo001mmm<br>disp      |
| OR reg, reg<br>OR reg, mem<br>OR mem, reg | 000010dw oorrrmmm<br>disp | RCL reg, imm<br>RCL mem, imm                 | 1100000w oo010mmm<br>disp data |
| OR reg, imm<br>OR mem, imm                | 1000000w oo001mmm<br>disp | RCR reg, imm<br>RCR mem, imm                 | 1100000w oo011mmm<br>disp data |
| POP reg                                   | 01011rrr                  | ROL reg, imm<br>ROL mem, imm                 | 1100000w oo000mmm<br>disp data |
| POP mem                                   | 10001111 oo000mmm<br>disp | ROR reg, imm<br>ROR mem, imm                 | 1100000w oo001mmm<br>disp data |
| PUSH reg                                  | 01010rrr                  | ADD reg, reg<br>ADD reg, mem<br>ADD mem, reg | 000000dw oorrrmmm<br>disp      |
| PUSH mem                                  | 11111111 oo110mmm<br>disp | SUB reg, reg<br>SUB reg, mem<br>SUB mem, reg | 001010dw oorrrmmm<br>disp      |
| PUSH imm                                  | 01101000 data             | LOOP Label                                   | 11100010 disp                  |
| INC reg<br>INC mem                        | 1111111w oo000mmm<br>disp | DEC reg<br>DEC mem                           | 1111111w oo001mmm<br>disp      |

| MOD = 11 |     |     | Direct Effective Address |                |                          |                           |
|----------|-----|-----|--------------------------|----------------|--------------------------|---------------------------|
| R/M      | W=0 | W=1 | R/M                      | MOD = 00       | MOD = 01                 | MOD = 10                  |
| 000      | AL  | AX  | 000                      | [BX]+[SI]      | [BX]+[SI]+D <sub>8</sub> | [BX]+[SI]+D <sub>16</sub> |
| 001      | CL  | CX  | 001                      | [BX]+[DI]      | [BX]+[DI]+D <sub>8</sub> | [BX]+[DI]+D <sub>16</sub> |
| 010      | DL  | DX  | 010                      | [BP]+[SI]      | [BP]+[SI]+D <sub>8</sub> | [BP]+[SI]+D <sub>16</sub> |
| 011      | BL  | BX  | 011                      | [BP]+[DI]      | [BP]+[DI]+D <sub>8</sub> | [BP]+[DI]+D <sub>16</sub> |
| 100      | AH  | SP  | 100                      | [SI]           | [SI]+D <sub>8</sub>      | [SI]+D <sub>16</sub>      |
| 101      | CH  | BP  | 101                      | [DI]           | [DI]+D <sub>8</sub>      | [DI]+D <sub>16</sub>      |
| 110      | DH  | SI  | 110                      | Direct Address | [BP]+D <sub>8</sub>      | [BP]+D <sub>16</sub>      |
| 111      | BH  | DI  | 111                      | [BX]           | [BX]+D <sub>8</sub>      | [BX]+D <sub>16</sub>      |